



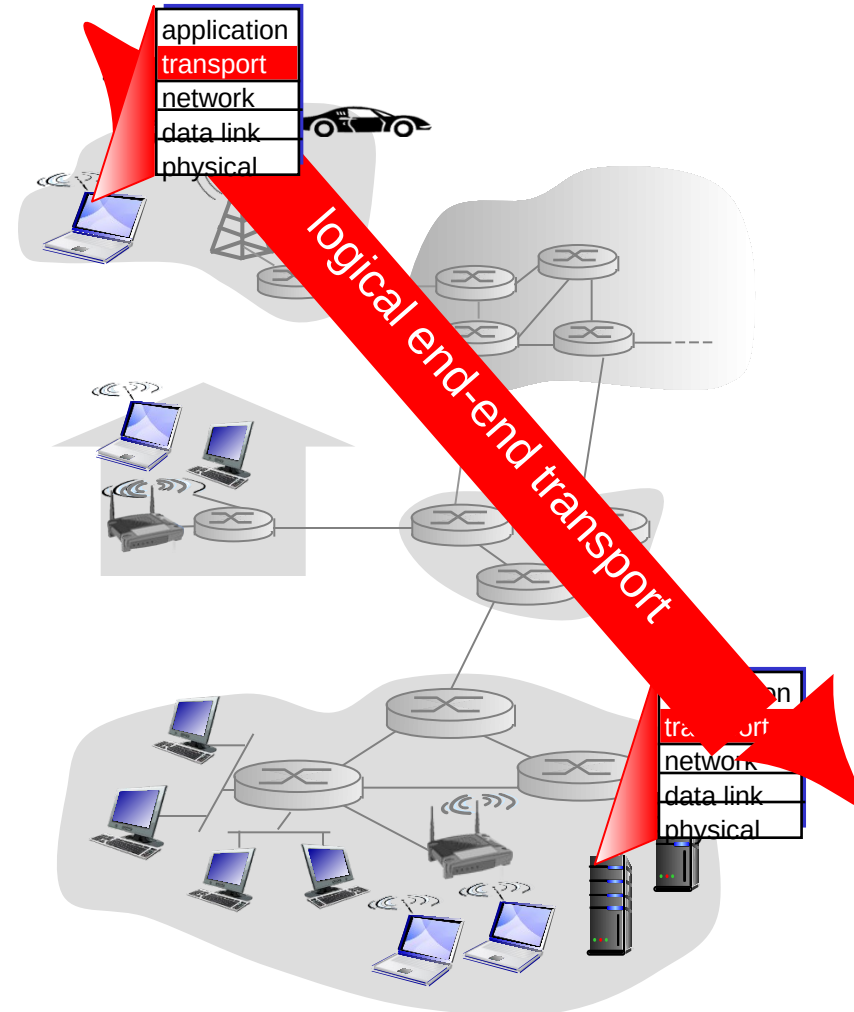
UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Livello Trasporto

Internet, Web e Cloud
Claudia Canali

Livello trasporto

- Offre **comunicazione logica** end-to-end tra i processi in esecuzione su host diversi
- Usato nei sistemi periferici
 - Lato mittente: **spezza** i messaggi dell'applicazione in **segmenti** e li passa al livello rete
 - Lato ricevente: **riassembla** i **segmenti** in messaggi, e li passa al livello applicazione
- Due protocolli disponibili
 - UDP (User Datagram Protocol)
 - TCP (Transmission Control Protocol)



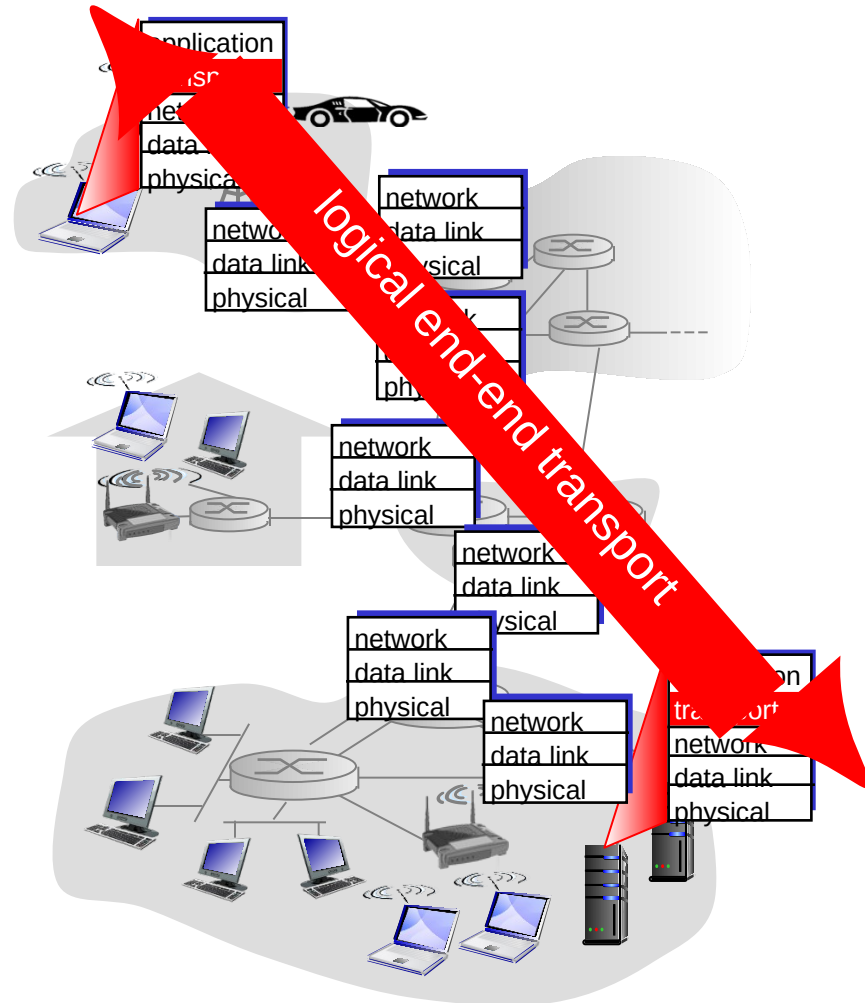
- **Livello trasporto:** comunicazione logica tra **processi**
- **Livello rete:** comunicazione logica tra **host**
- **Analogia dei condomini:** 10 ragazzi nel condominio di Anna inviano lettere a 10 ragazzi nel condominio di Marco
 - Host = condominio di Anna e condominio di Marco
 - Processi applicativi = ragazzi che si scambiano lettere
 - Messaggi applicativi = lettere
 - Protocollo di trasporto = Anna e Marco che raccolgono e smistano le lettere ai ragazzi nel loro condominio, e le imbucano nella cassetta della posta
 - Protocollo di rete = servizio postale

Panoramica livello di trasporto



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- Router Intermedi → **livello di rete (IP)**
 - Servizio **best-effort** delivery → **inaffidabile** - non offre garanzie (né consegna né ordine né integrità)
- Livello Trasporto → **estende servizi** di IP
- **UDP** (servizi minimi)
 - **Multiplexing/demultiplexing**
 - **Controllo di integrità** (riconoscimento di errori end-to-end)
- **TCP aggiunge** ai servizi minimi **affidabilità**:
 - **Garanzia sulla consegna (e ordine)**
 - **Controllo di flusso** (destinatario)
 - **Controllo di congestione** (rete)
- Servizi **non disponibili**:
 - Garanzie sul **ritardo**
 - Garanzie sulla **banda disponibile**
 - Garanzie sul **throughput**



Multiplexing/Demultiplexing

Multiplexing / demultiplexing

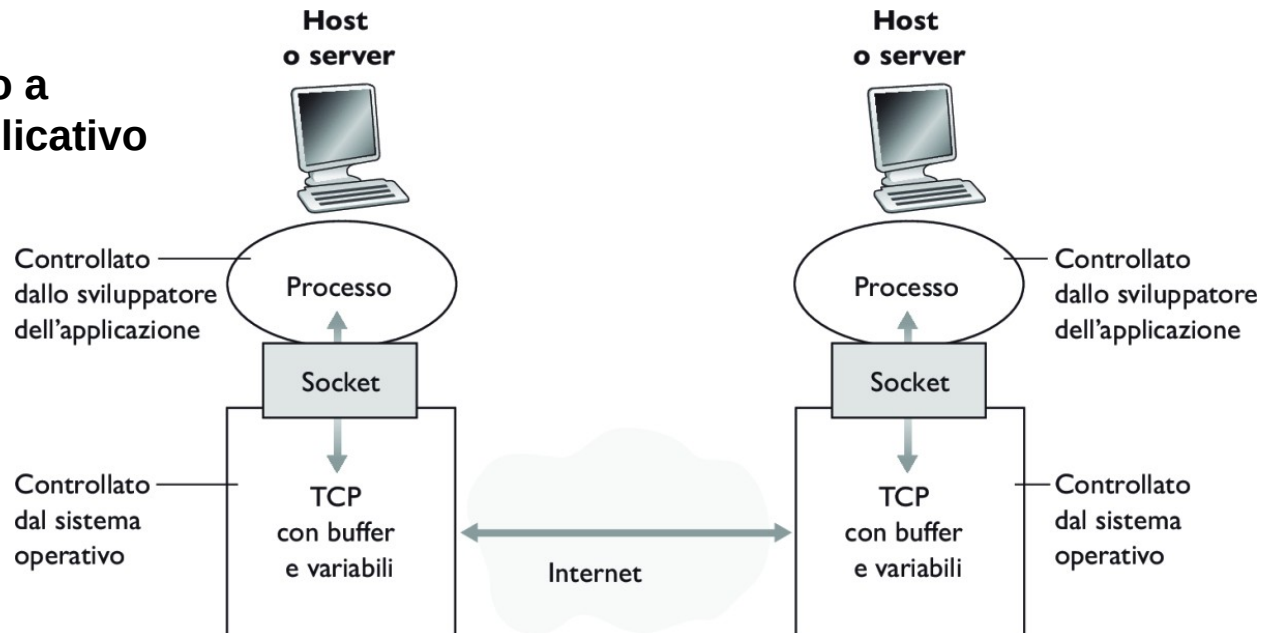


UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- **Tipicamente più processi** sono in esecuzione su un host
- Es. state scaricando pagine Web sul vostro computer da 2 browser diversi mentre sono in esecuzione 1 sessione FTP e 1 sessione Telnet
→ 4 processi applicativi in esecuzione
- Nell'host destinatario dei dati, il livello di trasporto riceve segmenti dal livello di rete
 - Compito di consegnare i segmenti al processo applicativo appropriato
- Interfaccia tra processo e rete: la **socket (o API tra applicazione e rete)**

Tutti i pacchetti arrivano con lo stesso IP di destinazione

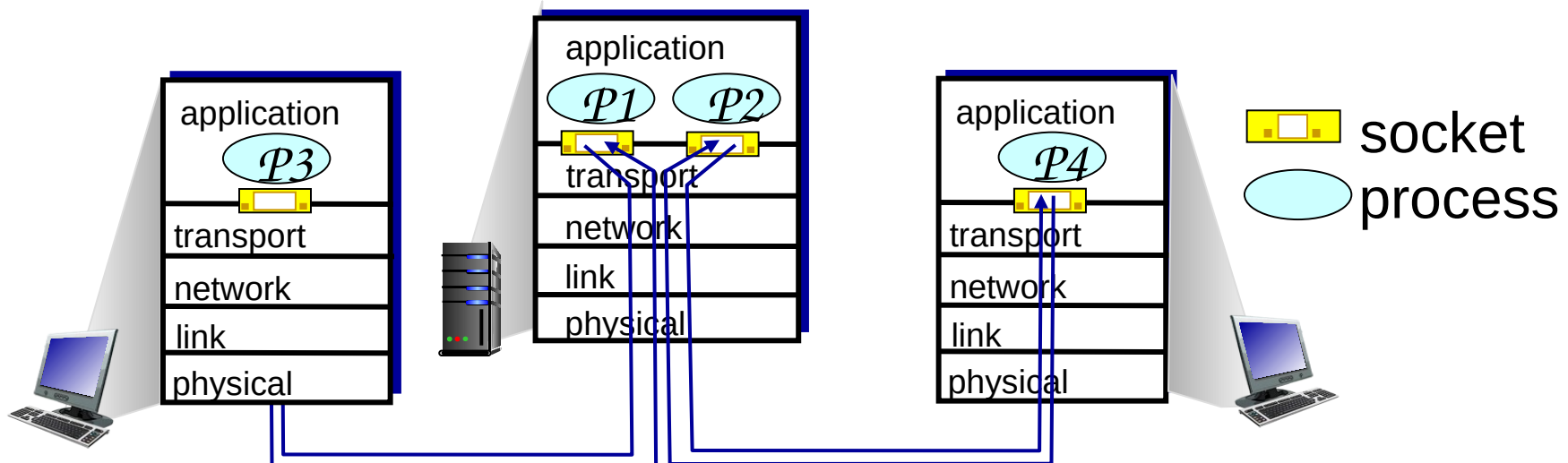
Come fa il livello di trasporto a capire a quale processo applicativo consegnare i pacchetti?



Multiplexing / demultiplexing



- **Multiplexing lato mittente:** il livello trasporto deve gestire dati in arrivo da più socket --> **incapsula i dati** con le intestazioni di livello trasporto (usate lato ricevente) per creare **segmenti** e passarli a **livello rete**
- **Demultiplexing lato ricevente:** uso degli header di intestazione per smistare i segmenti ricevuti alla **socket corretta** (socket come intermediario verso il processo destinatario)
- *Ma come identifico la socket? Quali informazioni servono?*
 - Ci aiutano i **numeri di porta** (ma non confondiamo porte con socket!)



Numeri di porta



Un numero di porta è identificato da un campo a **16 bit** che può assumere un **valore tra 0 e 65535**

E' possibile suddividere i tipi di porte in **tre categorie**:

- **Well Known o System Ports**: il cui valore va **da 0 a 1023** sono assegnate a specifici protocolli dalla Internet Assigned Number Authority (IANA)
 - Es. porta 80 per HTTP
- **User or Registered Ports**: il cui valore va da 1024 a 49151, utilizzabili dai protocolli di trasporto per identificare servizi di livello applicativo (*assegnabili da user – programmatori*)
- **Dynamic o Private Ports**: il cui valore va da 49152 a 65535, vengono assegnate dinamicamente, dal sistema operativo, tipicamente:
 - Quando un client si connette ad un host remoto (assegnate dinamicamente dal sistema operativo client)

Alcune Well-known ports



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Nei sistemi operativi derivati da UNIX ricevere connessioni su una porta nota richiede privilegi di root

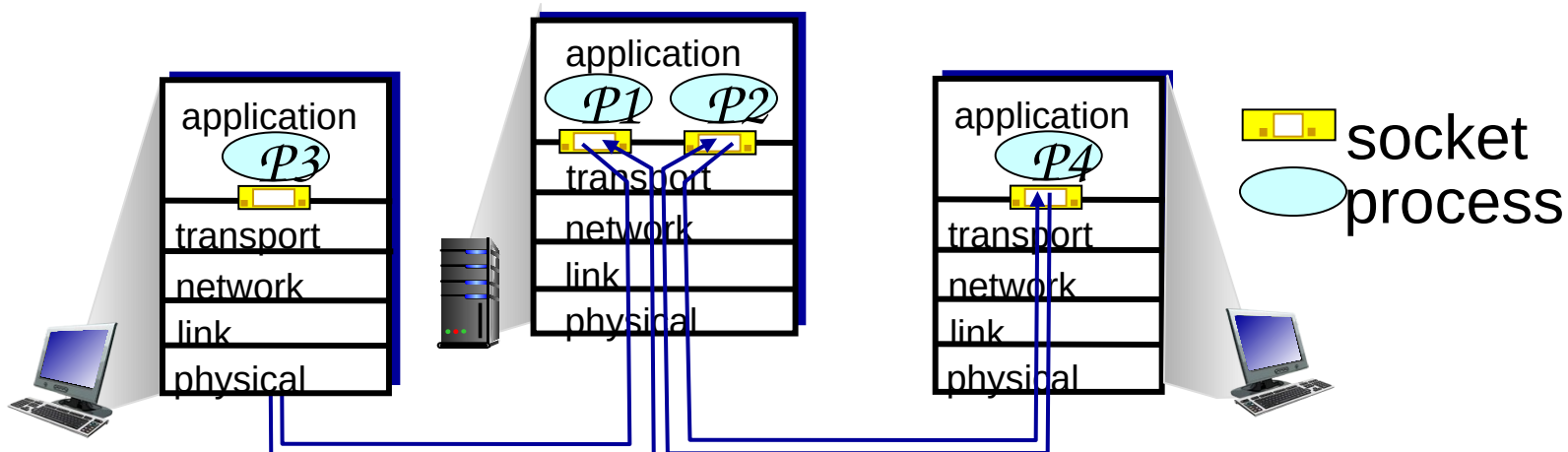
Port number	Assignment
20	File Transfer Protocol (FTP) Data Transfer
21	File Transfer Protocol (FTP) Command Control
22	Secure Shell (SSH) Secure Login
23	Telnet remote login service, unencrypted text messages
25	Simple Mail Transfer Protocol (SMTP) E-mail routing
53	Domain Name System (DNS) service
67, 68	Dynamic Host Configuration Protocol (DHCP)
80	Hypertext Transfer Protocol (HTTP) used in the World Wide Web
110	Post Office Protocol (POP3)
119	Network News Transfer Protocol (NNTP)
123	Network Time Protocol (NTP)
143	Internet Message Access Protocol (IMAP) Management of digital mail
161	Simple Network Management Protocol (SNMP)
194	Internet Relay Chat (IRC)
443	HTTP Secure (HTTPS) HTTP over TLS/SSL

Socket come endpoint

- **Endpoint** di una connessione TCP o UDP
→ *Lo scopo dei numeri di porta è **differenziare più endpoint (socket) su uno stesso indirizzo di rete (host)***
- **Comportamento diverso per UDP e TCP**

Per connessioni TCP:

- *Due host possono instaurare tra loro più di una connessione TCP*
- È la **coppia di socket** (nella pratica, la 4-upla costituita dall'indirizzo IP del client, dal numero di porta del client, dall'indirizzo IP del server e dal numero di porta del server) che **identifica in modo univoco ogni connessione TCP in una rete**



Funzionamento demultiplexing

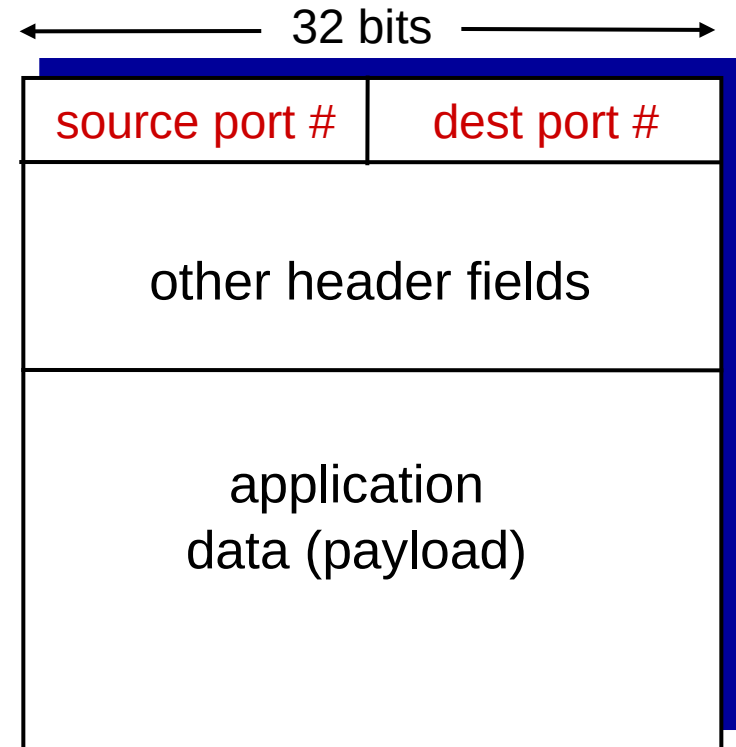


L'host riceve dalla rete i datagrammi IP

- **Ogni datagramma IP** contiene gli **indirizzi IP di sorgente e destinazione** (source IP address, destination IP address)
- Ogni datagramma trasporta un **segmento** di livello trasporto
- **Ogni segmento** contiene i **numeri di porta di mittente e destinazione** (source e destination port number)

L'host usa gli **indirizzi IP e i numeri di porta** per indirizzare i segmenti ricevuti alla giusta socket

Vediamo come UDP e TCP usano (in modo differente) questi dati...



Formato segmento livello trasporto (TCP/UDP)

Demultiplexing UDP: non orientato alla connessione



Vediamo qualche esempio a livello di socket programming

- **Socket UDP:** può essere creata con
`serversocket = socket(AF_INET, SOCK_DGRAM)`
 - Numero di porta assegnato automaticamente o bind su porta specifica da parte del programmatore: `serversocket.bind((' ', 12000))` ←
 - **Quindi ogni socket UDP è associata ad un numero di porta**
 - Ricordiamo: un segmento UDP contiene la porta del destinatario
 - Quando un host riceve un segmento UDP
 - Controlla il #porta destinazione nel segmento
 - Invia il segmento a quella porta (**demultiplexing**)
- ***una socket UDP è univocamente identificata da 2 elementi: indirizzo IP e numero di porta del destinatario***

Scelto tra le User Ports

NOTA: segmenti UDP che presentano diversi indirizzi IP e/o diversi numeri di porta di origine, ma hanno lo stesso indirizzo IP e lo stesso numero di porta di destinazione, vengono **diretti allo stesso processo di destinazione tramite la medesima socket**

Demultiplexing TCP: orientato alla connessione



- **Una socket TCP è identificata da 4 elementi**
 - Indirizzo IP del destinatario
 - Numero di porta del destinatario
 - Indirizzo IP del mittente
 - Numero di porta del mittente
- **Demultiplexing:** il ricevente usa **tutti i 4 valori** per indirizzare correttamente il segmento alla **socket appropriata**
- **Lo stesso processo server** (es. Web) sull'host destinatario può supportare **più socket TCP**
 - Tipicamente i server Web hanno **socket diverse per ogni client** che si connette (socket dedicate)
 - In caso di connessioni HTTP non persistenti si ha una **diversa socket per ogni richiesta**

NOTA: segmenti TCP in arrivo allo stesso destinatario con la stessa porta di destinazione che arrivano da mittenti diversi (indirizzi IP e/o porte diverse) vengono diretti a socket diverse!

Demultiplexing TCP: orientato alla connessione



- Tipicamente i server Web creano **socket diverse per ogni client** che si connette

```
server_address = "127.0.0.1"
```

```
serverPort = 12000 #80 sarebbe il default
```

```
serverSocket = socket(AF_INET, SOCK_STREAM)
```

```
serverSocket.bind((server_address, serverPort))
```

```
serverSocket.listen(1)
```

```
print('The server is ready to receive')
```

```
while True:
```

```
    connectionSocket, clientAddress = serverSocket.accept()
```

clientAddress è composto dall'indirizzo IP e
dal numero di porta del client

```
    sentence = connectionSocket.recv().decode()
```

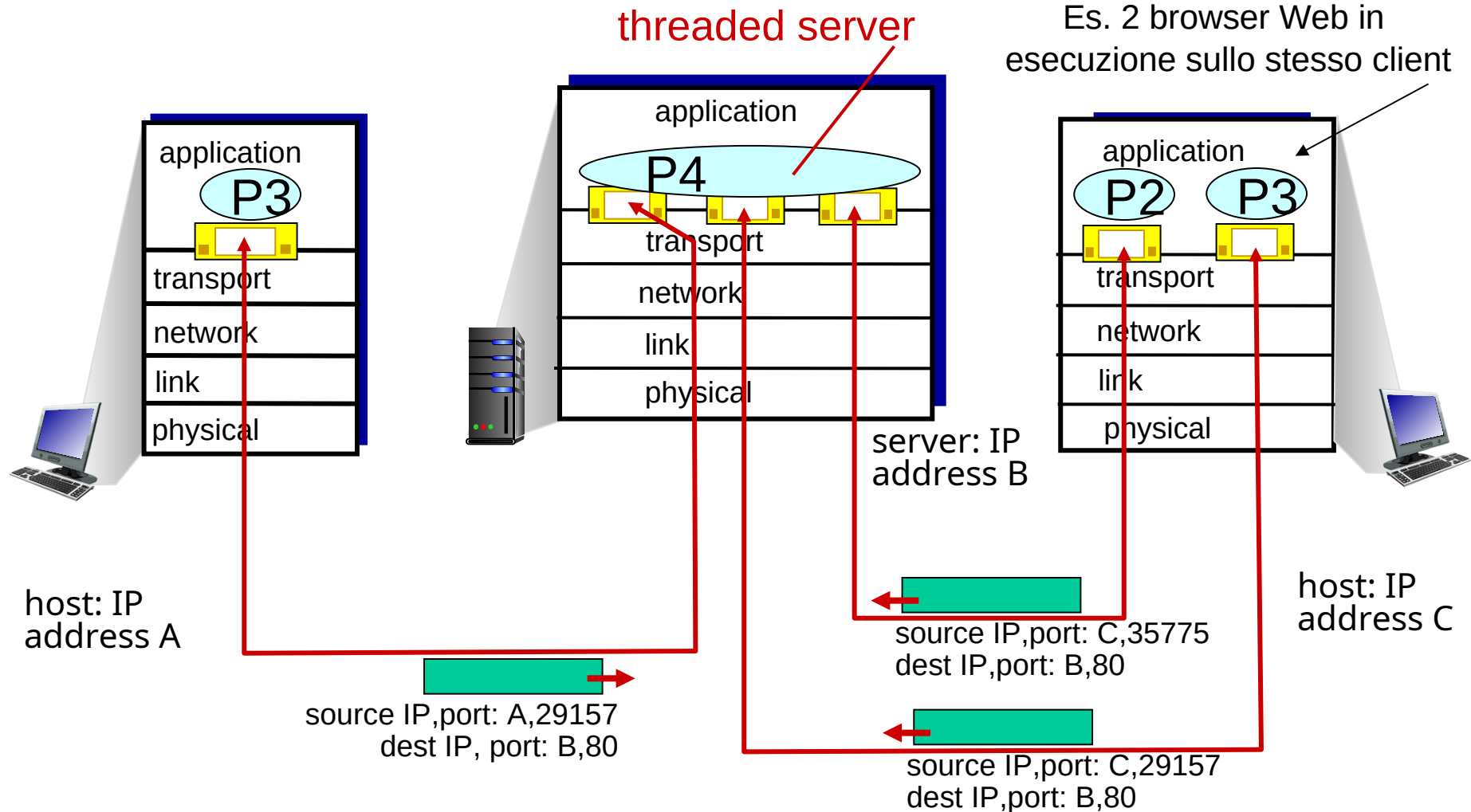
ServerSocket
(socket in ascolto
sulla porta 12000)

Il client invia una richiesta di connessione
(clientsocket.connect(serverName, 12000))

Il processo server crea
una nuova socket
(connectionSocket) su
cui si mette in attesa

La nuova **connectionSocket** è identificata da **4 valori**: IP e #porta mittente (client), IP e #porta server: è il livello di trasporto a tenere conto dell'associazione e fare demultiplexing

Demultiplexing TCP: esempio



UDP

User datagram Protocol

UDP: Caratteristiche



User Data Protocol (UDP), definito in [RFC 768]

Protocollo di trasporto leggero, dotato delle funzionalità minime del trasporto:

- **Servizio di moltiplicazione/demoltiplicazione**
 - UDP aggiunge al messaggio proveniente dal livello applicativo il numero di porta del mittente e del destinatario
- **Controllo di errore**
 - UDP include nell'header un campo di checksum

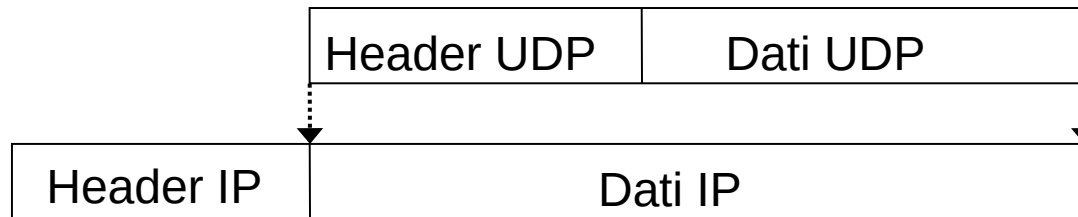
Cosa non ha

- **Servizio di consegna non garantito, ma solo di tipo best effort**
 - I segmenti UDP possono essere persi, duplicati, consegnati senza ordine
- **Servizio senza connessione (connectionless)**
 - Non vi è handshaking tra mittente e destinatario del segmento UDP
 - Ogni segmento UDP è trattato in **modo indipendente** dagli altri

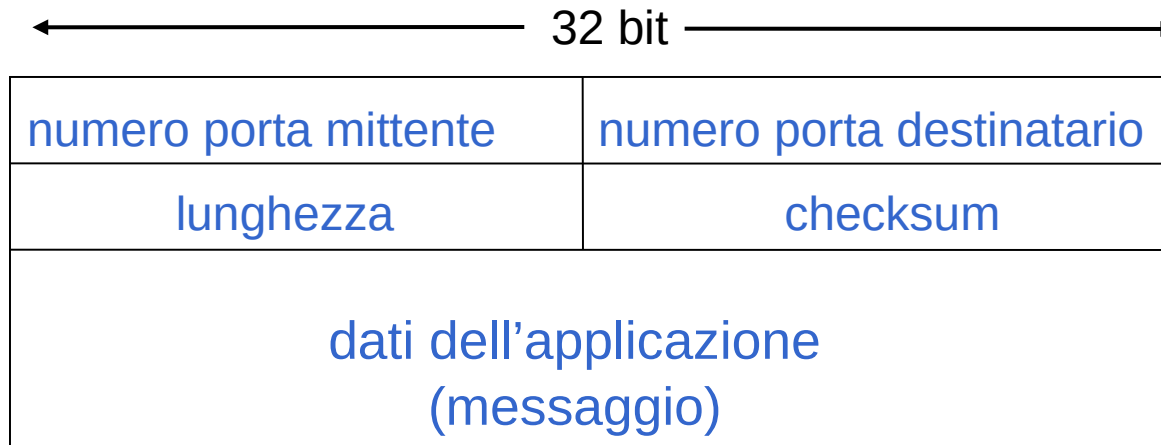
UDP: Formato segmento



Segmento UDP incapsulato in un datagramma IP



Segmento UDP



- Numero di porta del mittente (16 bit)
- Numero di porta del destinatario (16 bit)
- Lunghezza (16 bit): dimensione in byte del segmento
 - lunghezza = header + dati
- Header: dimensione pari a 8 byte

UDP: chi lo usa e perchè?



- DNS
- Ping (ICMP)
- Applicazioni di streaming multimediale (tolleranti alle perdite, sensibili al rate d'invio)
- SNMP (Simple Network Management Protocol) – livello rete

Perchè usarlo?

- **Non** richiede **instaurazione di connessione** – può aggiungere ritardo
- **Semplicità**: nessuno stato di connessione né al mittente né al destinatario
- Minor spazio usato per **l'intestazione** (8 byte intestazione UDP, 20 byte TCP)
- **Nessun controllo di congestione**: alcune applicazioni richiedono velocità minime di trasmissione (es. streaming, videoconferencing) → controllo più fine a livello di applicazione mittente sul rate dei dati inviati

Scopo: individuare “errori” (es., bit modificati/corrotti) nel segmento trasmesso

- Non è detto che tutti i link forniscano un servizio di livello Host-to-Network per rilevare errori di trasmissione (molti lo fanno)
- Checksum a livello IP limitato: se l'errore si verifica nella memoria di un router? (*principio end-to-end*)
- Non c'è recupero dell'errore (in alcune implementazioni il segmento viene scartato, in altre consegnato all'applicazione segnalando l'errore)

Mittente

- Tratta i contenuti del segmento (dati) come una sequenza di interi a 16 bit
- Checksum=complemento a 1 della somma dei contenuti del segmento
- Il mittente invia il valore del checksum nel campo checksum del segmento UDP

Destinatario

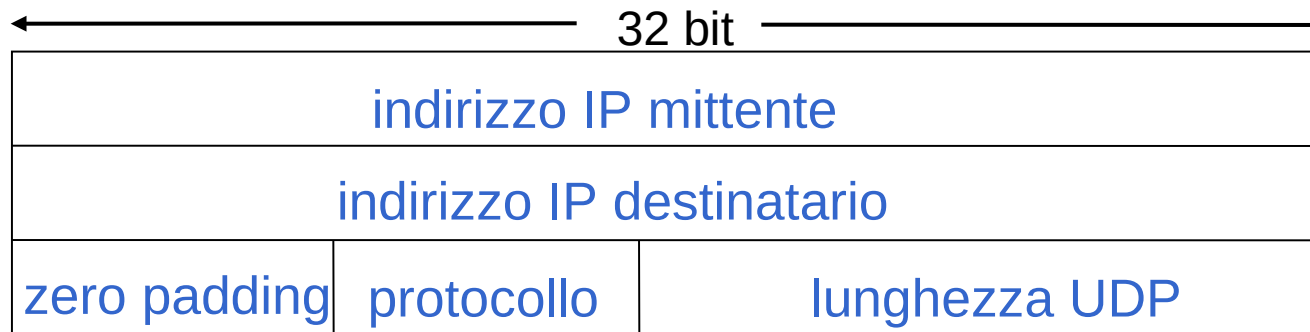
- Calcola il checksum del segmento ricevuto
- Controlla se il valore del checksum calcolato è uguale al valore del campo checksum:
 - NO → errore
 - SI → non si individua errore

UDP pseudo-header



In realtà il checksum viene calcolato usando un maggior numero di informazioni di quelle presenti nell'header UDP → definizione di uno ***pseudo-header* UDP che include gli indirizzi IP**

Obiettivo: ulteriore controllo sul fatto che il pacchetto abbia raggiunto la giusta destinazione



- *zero padding*: dimensione dello pseudo-header (multiplo di 32 bit)
- *protocollo*: campo protocollo del datagram IP
- pseudo-header anteposto al segmento UDP
- checksum calcolato su pseudo-header e intero segmento UDP
- lo pseudo-header *non* è trasmesso dal mittente

Checksum UDP



- Calcolato usando il complemento ad 1 della somma di tutti i campi dello pseudo-header e del segmento UDP

- Esempio

3 parole da 16 bit l'una

```
0110011001100110
0101010101010101
0000111100001111
```

Somma delle 3 parole

```
0110011001100110
0101010101010101
0000111100001111
-----
1100101011001010
```

- complemento ad 1 di 1100101011001010 → 0011010100110101
- campo checksum nel segmento UDP trasmesso = 0011010100110101
- il destinatario calcola il suo checksum su pseudo-header e segmento UDP ricevuto (senza calcolare il complemento a 1)
- checksum dest. + checksum UDP = 1111111111111111 → no errore
- checksum dest. + checksum UDP ≠ 1111111111111111 → errore

Costruzione pseudo-header



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- Necessaria la conoscenza dell'indirizzo IP del mittente e del destinatario
- Il destinatario a livello UDP non può acquisire l'indirizzo IP del mittente dall'applicazione di livello superiore
- Il processo destinatario a livello UDP chiede al livello IP di costruire lo pseudo-header per il calcolo del checksum UDP

→ *Violazione del principio di indipendenza funzionale per protocolli appartenenti a livelli diversi*

"E' vietata la copia e la riproduzione dei contenuti e immagini in qualsiasi forma.

E' inoltre vietata la redistribuzione e la pubblicazione dei contenuti e immagini non autorizzata espressamente dall'autore o dall'Università di Modena e Reggio Emilia."